

ADVANCED COMPUTER ARCHITECTURE

Francesco Leporati – Emanuele Torti

Tel: 0382 985678/371

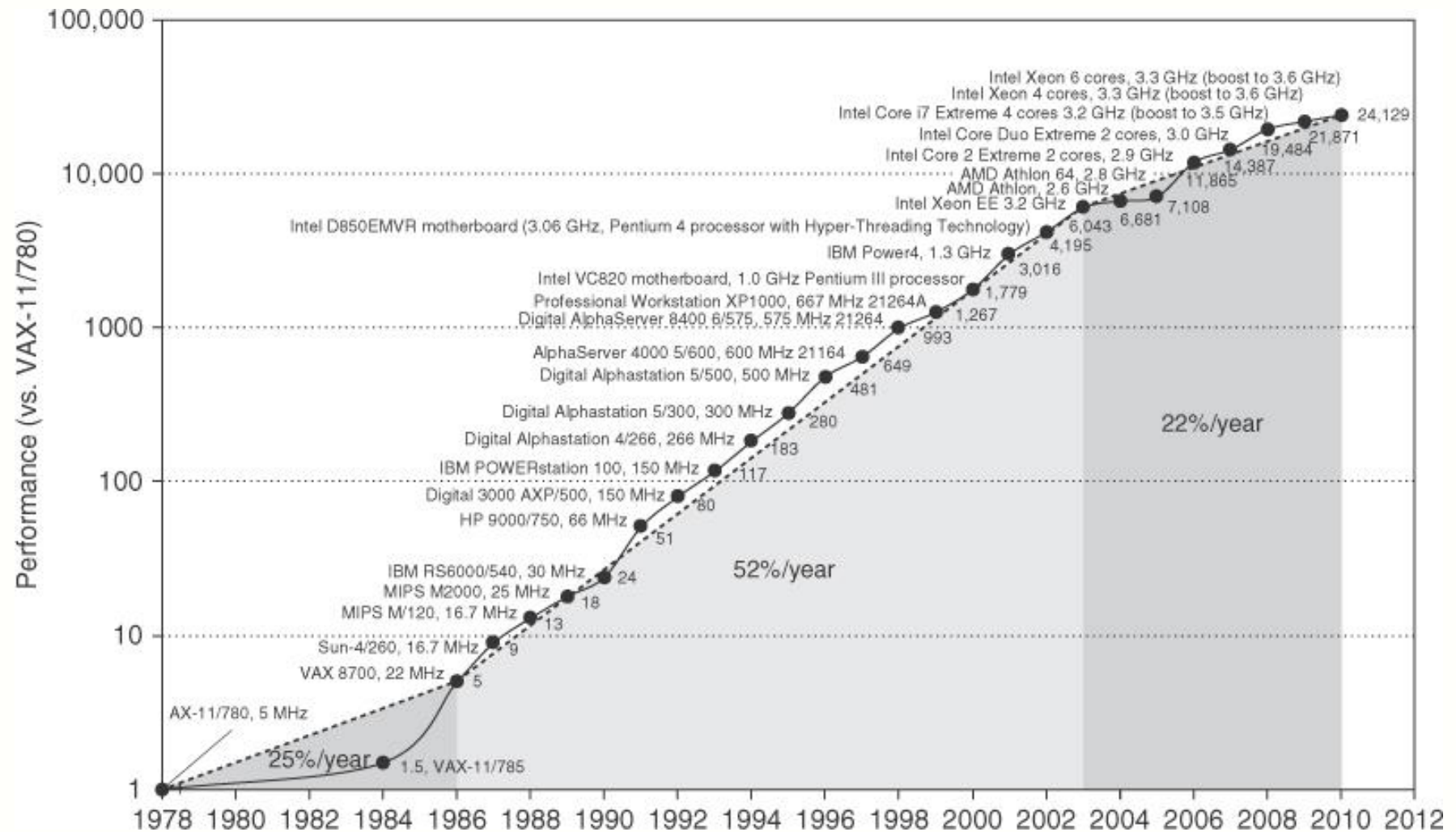
E-mail: francesco.leporati@unipv.it
 emanuele.torti@unipv.it

Course syllabus and motivations

- This course covers the **internal architecture** and operation of modern processors, and the **fundamentals of parallel programming**.
- The increase in performance of processors in the last years is mainly due to three factors:
 - speed-up in instruction execution
(what is actually the speed of execution, how can it be measured?)
 - increase in the number of instructions executed in parallel
(given a set of instructions, which are the properties they must satisfy to be executed in parallel?)
 - increase in the number of processors that are embedded in the same chip/board and that can be programmed to execute an algorithm

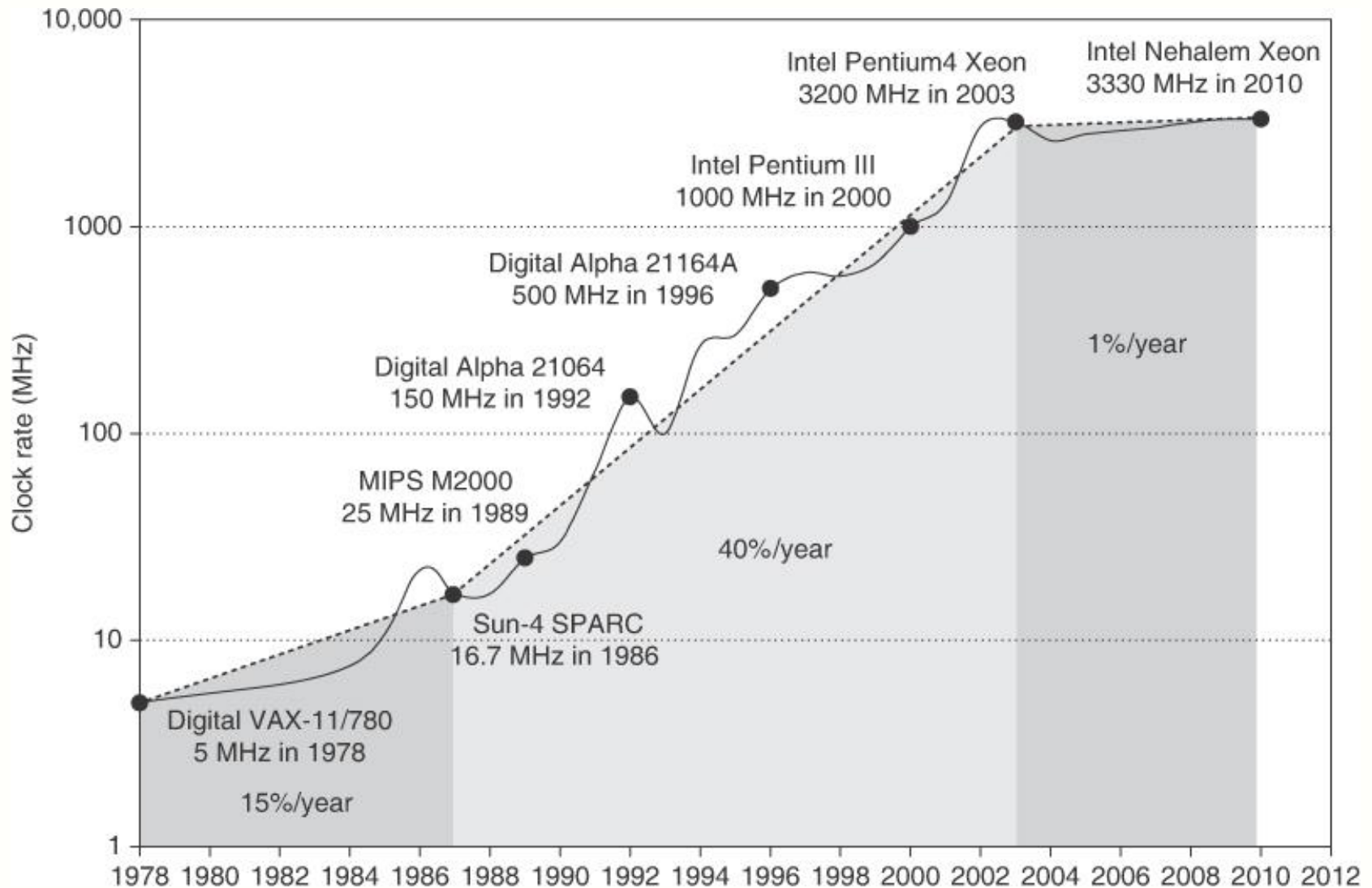
Course syllabus and motivations

- Increase in Cpu performance from 1978 to 2010 (Fig. 1.1 H-P5)



Course syllabus and motivations

- Increase in clock speed 1978 to 2010 (Fig. 1.1 H-P5)



Course syllabus and motivations

- Since 2003 this trend has slowed down indeed (can you guess why?)

Critical issues

- In the last 30 years the clock passed from 1 MHz to 4 GHz but it cannot grow up indefinitely
- The dynamic power in a CMOS circuit is $\propto$ to f^3 -> problems of power dissipation

$P = \alpha C_L V^2 f$ with α switch probability, C_L capacity, V voltage ($\approx V^2$), f frequency

- Moore law in the recent years does not perform well as before
- Multicore processors to increase computational capability ...
- ... but there is an intrinsic dimensional limit to frequency augmentation

If c (light velocity) = 300.000 Km/sec = $3 * 10^9$ dm/sec and $f = 4$ GHz

$$s = c * t = 3 * 0.25 = 0.75 \text{ dm}$$

Course syllabus and motivations

- Designers have tried alternative approaches: using multiple execution units. This can be realized in a few ways, also with a mixed approach:
 - Multithreading
 - Multi-core processors, enhanced integration with GPUs
 - Shared-memory multiprocessors systems
 - Multiprocessors with partitioned memory
- (Question: these solutions require a novel approach compared to a uniprocessor, can you figure out?)

Course syllabus (A)

- **Part Leporati:**
 - Fundamentals in modern architectures
 - Pipelining basics
 - Instruction Level Parallelism (ILP): the DYNAMIC/STATIC approach
 - Multithreading basics
- **Part Torti:**
 - Memory hierarchy design
 - Cache and coherence
 - Virtual memory
 - Performance modeling
 - Storage systems
 - I/O architecture
- **Project:**
 - parallel programming with GPU CUDA (laboratory)

Books

- **J. L. Hennessy & D. A. Patterson:** *Computer Architecture: A Quantitative Approach*, Elsevier - Morgan Kaufmann
 - 3rd ed. 2003: in the charts as “Hennessy-Patterson”
 - 4th ed. 2007: in the charts as “H-P4”
 - 5th ed. 2011: in the charts as “H-P5”
- **D. A. Patterson & J. L. Hennessy :** *Computer Organization and Design, The Hardware/Software Interface*, third edition, 2005
 - in the charts as “Patterson-Hennessy”
- **A. S. Tanenbaum:** *Structured Computer Organization*, 5th ed. Pearson, 2005

Books – Italian editions

- **J. L. Hennessy & D. A. Patterson:** *Architettura degli elaboratori* Apogeo, 2008 (quarta edizione americana)
 - 4th ed. 2007: in the charts as “H-P4”
- **D. A. Patterson & J. L. Hennessy :** *Struttura e Progetto dei Calcolatori: l'interfaccia Hardware-software*, seconda edizione (terza edizione americana) Zanichelli, 2006
- **D. A. Patterson & J. L. Hennessy :** *Struttura e Progetto dei Calcolatori: terza edizione* (quarta edizione americana) Zanichelli, 2010
- **A. S. Tanenbaum:** *Architettura dei Calcolatori, un approccio strutturale*, 5^t ed. Pearson/Prentice Hall, 2006

Further material

- Most recent architectures are not properly described in textbooks. The course uses papers, benchmarks and graphical material drawn from web resources, mainly from
- **INTEL**: <http://www.intel.com/technology/ITJ/>
- **Wikipedia**: en.wikipedia.org
- **Anandtech**: www.anandtech.com
- **Tom's Hardware**: www.tomshardware.com
- **Hardware Upgrade**: www.hwupgrade.it

Lessons charts

- Download charts used during class
- Use the charts as a trace of the arguments
- Textbook are the course reference material; detailed instructions on which textbooks and which chapters cover the exam will be published in due course on the web site

Course entry requirements

- Basic knowledge of the “Von-Neumann” CPU and of virtual memory
- Basic knowledge of operating systems (processes, thread, mutual exclusion, synchronization)
- Elementary knowledge of assembly language
- Knowledge of the C-language

Testing and exams

- Parallel (GPU) programming project (optional)
 - assigned individually, developed on personal PC but running on UNIPV machines
 - individual discussion of the project
- Written test
 - individual
- Maximum grade without project: 26

Course web site

kiro

Course outline

The architecture from the programmer's view point

10000x10000 array, Intel Core 2 Duo @ 2.8 GHz

```
int sum1(int** m, int n) {  
    int i,j,sum=0;  
    for (i=0; i<n;i++)  
        for (j=0; j<n; j++)  
            sum += m[i][j];  
    return sum;  
}
```

0.4 seconds

```
int sum2(int** m, int n) {  
    int i,j,sum=0;  
    for (i=0; i<n;i++)  
        for (j=0; j<n; j++)  
            sum += m[j][i];  
    return sum;  
}
```

1,7 seconds

(4.2 times slower !!)

Course outline

- Understanding the operation of modern architectures requires a little bit of analysis of their evolution
- All modern processor have roughly the same operation mode, and quite similar internal architecture, the so-called **RISC architecture**
- This denomination is almost forgot, it was very much in use in the 80's, it marked the distinction between the “old” **CISC architectures** and the “new” ones, RISC, that allowed to reach higher performances.

Course outline

- RISC architecture are simpler (and nicer) in concept, which allowed to deploy some fundamentals techniques to speed-up program execution:
 - pipelining (partially overlaps instruction execution)
 - parallel execution of independent instructions
 - branch prediction (knowing in advance which instruction to fetch)
 - speculation (instruction executed before knowing if they belong to the correct path, with possible “undoing”)
 - compile-time instruction re-ordering (static schedulation) to maximize processor utilization.

Course outline

- These techniques are used in all modern architectures, though their actual implementation differ from processor to processor
- After the major “CISC to RISC” step, performance increase has been due mainly to technologies, rather than to new concepts (to be debated)
- A single core in a dual/quad core Intel CPU is still very similar to an “old” Pentium I

Course outline

- Fabrication improvements however have almost stopped from 2003 onwards, so that uniprocessor performances have had no sensible increase any longer.
- The requirement of more and more computational power has led to alternative approaches.
- The basic idea was coupling two or more processors (“core”) in a multi-core architecture: modern dual and quad-core processors are small shared (and cache) multiprocessor systems.

Course outline

- Two or even more programs can be executed in parallel on the same multi-core processor (provided software can actually exploit available cores)
- Nothing new! this is the architecture of shared-memory multiprocessor systems (that featured private caches, yet)
- And even before multi-core processors, multi-threading tried already to carry out parallel execution of instructions belonging to different threads

Course outline

- If even more computational power is required, it is unfeasible using processors that share the same main memory
- It is mandatory using partitioned-memory multiprocessors, even thousands of them, each possibly a multi-core.
- Modern multiprocessors featuring the highest performances can be a synthesis of different architectural solutions: hundreds or thousands of computational units, linked with an ad-hoc network, each computational unit hosting multiple cores that share main memory, each core itself capable of multi-threading.

Course outline

- Old “alternative” architectures, such as vector processors, have almost completely disappeared.
- They left as their heritage multi-media SIMD instructions currently embedded in all processors, and in graphics processors (GPU).
- Currently, the development environments support more and more GPU, even in general-purpose, non-graphicals applications.